

JDBC & Database Exercises (Merged & Translated)

Exercise 1 — Employee Database (JDBC + MySQL)

Goal: Use Java JDBC to perform basic operations on a MySQL database.

1. Create the database and table in MySQL

- Create a database named EmployeeDB.
- Create a table Employees with the following columns: id (int, auto-increment, primary key), name (varchar(50)), age (int), department (varchar(50)), salary (double).

2. Write a Java program that implements the following functions

- Connect to the EmployeeDB database.
- Add a new employee to the Employees table.
- Display the list of all employees.
- Update an employee's information by id.
- Delete an employee by id.
- Search for an employee by name or department.

Exercise 2 — Product Database (JDBC + MySQL)

Create the database and table in MySQL

1. Create a database named ProductDB.
2. Create a table Products with the following columns: id (int, auto-increment, primary key), name (varchar(100)), category (varchar(50)), price (double), quantity (int).

Write a Java program that implements the following functions

1. Connect to the ProductDB database.
2. Add a new product to the Products table.
3. Display the list of all products.
4. Update a product's information by id.
5. Delete a product by id.
6. Search for a product by name or category.

Requirements

- The in-memory data structure used to hold products before printing must use a Hashtable, with the key being the product's id and the value being the product object.
- Advanced (bonus): implement the program using the DAO pattern (model, repository, and implementation layers).

Exercise 3 — Employee Management Application (Design Pattern & Internationalization)

Write an employee management application with the following requirements.

Create the EmployeeDB database with the nhanvien table

```
tblNhanVien
int id
nvarchar(64) name
int yearOfBirth
varchar(256) address
varchar(11) phone
int salary
```

The program should have the following menu

```
===== MENU =====
1. Add new employee
2. List employees
3. Search employee
4. Change language (Vietnamese - English)
5. Exit
=====
```

Functional requirements

1. Design a Connection class using the Singleton pattern to create the database connection class.
2. Apply the DAO (Data Access Object) pattern to implement CRUD operations for employee management.
3. Use Locale and Resource Bundle to support multiple languages: Vietnamese and English.

Exercise 4 — Advanced JDBC: Goods (Inventory) Management

Write a goods (inventory) management program with the following information.

The Products table includes

1. Product code
2. Product name
3. Manufacturer
4. Quantity (in stock)
5. Original price (VND)
6. VAT tax

Program menu

```
===== MENU =====
1. Stock in (receive goods)
2. Stock out (issue goods)
3. Inventory
4. Update
5. Exit
===== ***** =====
```

Functional requirements

- Build a goods management application that applies advanced JDBC knowledge (batch inserts, using PreparedStatement, Stored Procedures).
- Stocking in and updating goods must use PreparedStatement to build queries.
- The “Inventory” option displays the list of all current products with full information.
- When stocking out, the quantity in stock must be decreased; if the quantity in stock is insufficient, a warning must be displayed and the transaction canceled (use a stored procedure to enforce this).

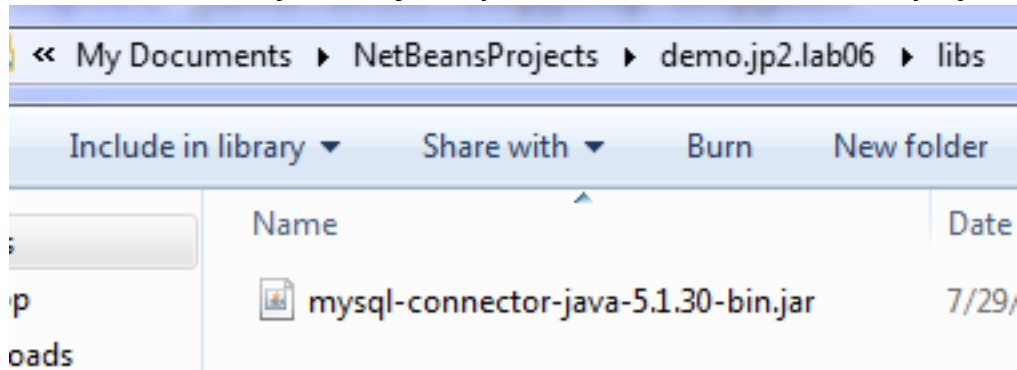
Exercise 5 — Student Management Console Application (JDBC)

Build a student-management console application, against a database named *qlsv* with a table *tblSinhVien* (columns: id, rollnumber, name, address, phone, gender).

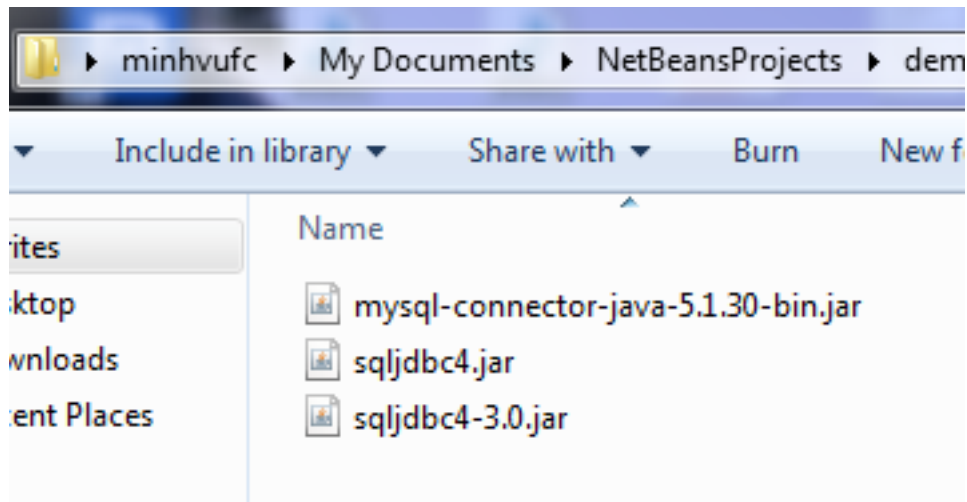
Part 1 — Connect to the Database and Report Connection Status

Create a database named *qlsv* to manage students. Create a Java application that connects to this database and displays a message indicating whether the connection succeeded or failed.

Step 1: Download the JDBC jar library and place it in the libs folder of the project



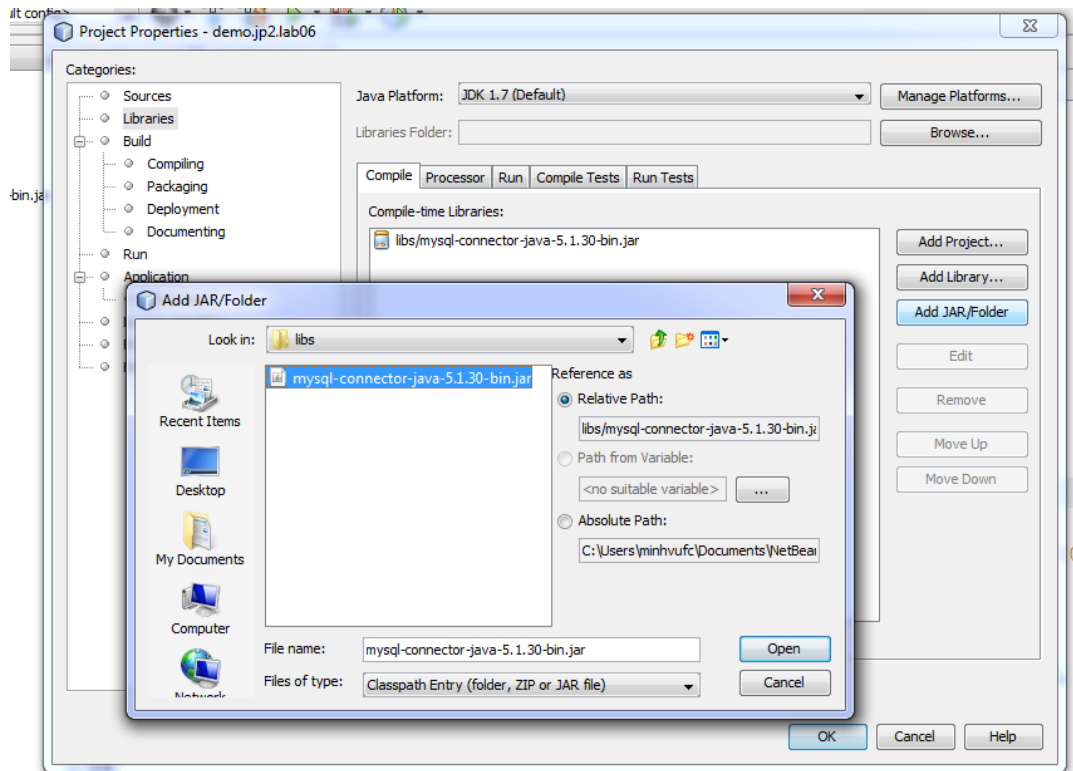
MySQL Server — mysql-connector-java-5.1.30-bin.jar placed in the libs folder.



Microsoft SQL Server — mysql-connector-java, sqljdbc4.jar and sqljdbc4-3.0.jar in the libs folder.

Step 2: Import the JDBC library for MySQL

Right-click the project, choose Properties, go to the Libraries node, click Add JAR/Folder, and browse to the jar file in the project's libs folder from Step 1.



Adding the JDBC driver jar to the project's compile-time libraries via Project Properties → Libraries → Add JAR/Folder.

Step 3: Create the *qlsv* database and the *tblSinhVien* table in MySQL

#	Name	Type	Collation	Attributes	Null	Default	Extra
1	id	int(11)			No	None	AUTO_INCREMENT
2	rollnumber	varchar(8)	utf8_general_ci		No	None	
3	name	varchar(256)	utf8_general_ci		No	None	
4	address	varchar(512)	utf8_general_ci		Yes	NULL	
5	phone	varchar(11)	utf8_general_ci		Yes	NULL	
6	gender	bit(1)			No	None	

Structure of the *tblSinhVien* table in the *qlsv* database: *id* (int, auto-increment), *rollnumber* (varchar(8)), *name* (varchar(256)), *address* (varchar(512), nullable), *phone* (varchar(11), nullable), *gender* (bit(1)).

Step 4: Declare a *MainClass* with a *main()* method and write the following code

```
package demo.jp2.lab06;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 *
 * @author minhvuvc
 */
public class MainClass {
```

```

/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    try {
        Class.forName("com.mysql.jdbc.Driver");
        Connection conn = null;
        conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/qlsv",
            "root", "");
        conn.close();
        System.out.println("Connected to the database successfully");
    } catch (ClassNotFoundException | SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
}
}

```

Note: MySQL installed locally has the address localhost, default port 3306, default username root and an empty password.

MS SQL installed locally usually has the address <machine name>/<server name>. It is called and used similarly to MySQL. See the appendix at the end of this guide for more details.

Step 5: Run the program and check the result — if unsuccessful, an exception message will be shown

```

Output - demo.jp2.lab06 (run)
Jul 29, 2015 2:19:31 AM demo.jp2.lab06.MainClass main
SEVERE: null
com.mysql.jdbc.exceptions.jdbc4.MySQLSyntaxErrorException: Unknown database 'qlsv'
    at sun.reflect.NativeConstructorAccessorImpl.newInstance0(Native Method)
    at sun.reflect.NativeConstructorAccessorImpl.newInstance(NativeConstructorAccessorImpl.java:57)
    at sun.reflect.DelegatingConstructorAccessorImpl.newInstance(DelegatingConstructorAccessorImpl.java:45)
    at java.lang.reflect.Constructor.newInstance(Constructor.java:526)
    at com.mysql.jdbc.Util.handleNewInstance(Util.java:409)
    at com.mysql.jdbc.Util.getInstance(Util.java:384)
    at com.mysql.jdbc.SQLError.createSQLException(SQLError.java:1052)
    at com.mysql.jdbc.MySqlIO.checkErrorPacket(MySqlIO.java:4232)
    at com.mysql.jdbc.MySqlIO.checkErrorPacket(MySqlIO.java:4164)
    at com.mysql.jdbc.MySqlIO.checkErrorPacket(MySqlIO.java:926)
    at com.mysql.jdbc.MySqlIO.proceedHandshakeWithPluggableAuthentication(MySqlIO.java:1748)
    at com.mysql.jdbc.MySqlIO.doHandshake(MySqlIO.java:1288)
    at com.mysql.jdbc.ConnectionImpl.coreConnect(ConnectionImpl.java:2506)
    at com.mysql.jdbc.ConnectionImpl.connectToServer(ConnectionImpl.java:2520)

```

Running the program before the qlsv database exists results in a MySQLSyntaxErrorException: Unknown database 'qlsv'.

```

Output - demo.jp2.lab06 (run)
run:
Kết nối tới CSDL thành công
BUILD SUCCESSFUL (total time: 0 seconds)

```

After creating the qlsv database, the program prints “Connected to the database successfully” and the build succeeds.

Part 2 — Insert Student Records from Keyboard Input

In the student management application above, write a function to input student information from the keyboard, then insert the entered data into the database using an SQL command with a Statement object.

Step 1: Code the SinhVien.java class

```
package demo.jp2.lab06;

import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;

/**
 *
 * @author minhvufo
 */
public class SinhVien {

    int id;
    String rollNumber;
    String name;
    String address;
    String phoneNumber;
    boolean gender;
}
```

Step 2: Write the input function

```
private void enterInfo() {
    System.out.println("Enter student information:");
    Scanner input = new Scanner(System.in);
    System.out.println("Student ID: ");
    this.rollNumber = input.nextLine();
    System.out.println("Full name: ");
    this.name = input.nextLine();
    System.out.println("Address: ");
    this.address = input.nextLine();
    System.out.println("Phone number: ");
    this.phoneNumber = input.nextLine();
    System.out.println("Gender (Male = 1 | Female = 0): ");
    int gend = input.nextInt();
    if (gend == 1) {
        this.gender = true;
    } else {
        this.gender = false;
    }
}
```

Step 3: Write the insert() function

```
public void insert(Statement st) throws SQLException {
    enterInfo();
    String sql = "INSERT INTO `tblsinhvien`("
        + "`rollnumber`, `name`, `address`, `phone`, `gender`)"
        + "VALUES ("
        + "'" + this.rollNumber + "',"
        + "'" + this.name + "',"
        + "'" + this.address + "',"
        + "'" + this.phoneNumber + "',"
        + "'" + this.gender + "'"
        + ")";
    st.executeUpdate(sql);
}
```

```

        + "'" + this.address + "',"
        + "'" + this.phoneNumber + "',"
        + "'" + this.gender + ")";
int row = st.executeUpdate(sql);
if (row > 0) {
    System.out.println("Added successfully");
} else {
    System.out.println("Failed, please check your data");
}
}
}

```

Step 4: Create a class named MainClass with a main() method, and write the menu-display function

```

package demo.jp2.lab06;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.Scanner;
import java.util.logging.Level;
import java.util.logging.Logger;

/**
 *
 * @author minhvufo
 */
public class MainClass {

    Connection conn = null;

    public void connect() {
        try {
            Class.forName("com.mysql.jdbc.Driver");
            conn = DriverManager.getConnection("jdbc:mysql://localhost:3306/qlsv",
                "root", "");
            System.out.println("Connected to the database successfully");
        } catch (ClassNotFoundException | SQLException ex) {
            Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
        }
    }

    private int showMenu() {
        System.out.println("===== MENU =====");
        System.out.println("1. Enter student information");
        System.out.println("2. List of students");
        System.out.println("3. Edit student");
        System.out.println("4. Delete student");
        System.out.println("5. Exit");
        System.out.println("Please choose from 1 -> 5");
        Scanner input = new Scanner(System.in);
        return input.nextInt();
    }

    /**
     * @param args the command line arguments
     */
}

```

```

    */
    public static void main(String[] args) {
    }
}

```

Step 5: Write the body of the main() function

```

/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    MainClass main = new MainClass();
    main.connect();
    int choose = 0;
    do {
        choose = main.showMenu();
        Statement st = null;
        try {
            st = main.conn.createStatement();
        } catch (SQLException ex) {
            Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
        }
        switch (choose) {
            case 1:
                SinhVien sv = new SinhVien();
                try {
                    sv.insert(st);
                } catch (SQLException ex) {
                    Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null,
ex);
                }
                break;
            case 2:
                break;
            case 3:
                break;
            case 4:
                break;
        }
    } while (choose > 0 && choose < 5);
    try {
        // Close the connection
        main.conn.close();
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
}

```

Step 6: Run the program and check the result


```
Output - demo.jp2.lab06 (run) #2

Vui lòng chọn từ 1 -> 5
1
Nhập thông tin sinh viên:
Mã sinh viên:
JF150801
Họ và Tên:
Jang Hang Ra
Địa chỉ:
Seoul
SĐT:
1234567890
Giới tính(Nam = 1 | Nữ = 0):
0
===== MENU =====
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5
|
```

Console output after choosing option 1 and entering a new student's information (ID JF150801, Jang Hang Ra, Seoul, 1234567890, Female).

+ Options

	id	rollnumber	name	address	phone	gender
☐ Edit Copy Delete	1	C1505001	Ma Sieu	Tay Luong	19001265	1
☐ Edit Copy Delete	2	C1505002	Dieu Thuyen	To Chau	19001001	0
☐ Edit Copy Delete	5	C1505003	Ma Sieu	Tay Luong	19001265	1
☒ Edit Copy Delete	6	JF150801	Jang Hang Ra	Seoul	1234567890	0

↑ ☐ Check All With selected: ☐ Change ☐ Delete ☐ Export

Number of rows: 25

The database table after the insert, showing the new row (id 6, JF150801, Jang Hang Ra, Seoul, 1234567890) highlighted.

Part 3 — Load and Display the List of Students

In the student management application above, write a function to load the list of students from the database and display it on the screen.

Step 1: Write the load() function

```
public void load(Statement st) throws SQLException {
    String sql = "SELECT * FROM `tblsinhvien`";
    ResultSet result = st.executeQuery(sql);
    System.out.println("-----");
    while (result.next()) {
        String id = result.getString("id");
        String rollnumber = result.getString("rollnumber");
        String name = result.getString("name");
        String address = result.getString("address");
        String phone = result.getString("phone");
        String gender = "Male";
        if (result.getInt("gender") == 0) {
            gender = "Female";
        }
        System.out.println(id + "| " + rollnumber + "| " + name + " - " + address + " - " + phone + " - " + gender);
    }
    System.out.println("-----");
}
```

Step 2: Call it from main()

```
case 2:
    try {
        sv.load(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
    break;
```

Step 3: Run the program and check the result

```
-----
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5
2
-----

1| C1505001| Ma Sieu - Tay Luong - 19001265 - Nam
2| C1505002| Dieu Thuyen - To Chau - 19001001 - Nữ
5| C1505003| Ma Sieu - Tay Luong - 19001265 - Nam
6| JF150801| Jang Hang Ra - Seoul - 1234567890 - Nữ
-----

===== MENU =====
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5
|
```

Console output after choosing option 2, listing all students currently in the database.

Part 4 — Edit and Delete Student Records

In the student management application above, write functions to edit a student's information and to delete a student's data, both identified by the student's RollNumber.

Step 1: Write the function to input updated information

```
private void editInfo() {
    System.out.println("Enter updated information:");
    Scanner input = new Scanner(System.in);
    System.out.println("Full name: ");
    this.name = input.nextLine();
    System.out.println("Address:");
    this.address = input.nextLine();
    System.out.println("Phone number: ");
    this.phoneNumber = input.nextLine();
    System.out.println("Gender (Male = 1 | Female = 0): ");
    int gend = input.nextInt();
    if (gend == 1) {
        this.gender = true;
    } else {
        this.gender = false;
    }
}
```

Step 2: Write the update() function

```
public void update(Statement st) throws SQLException {
    System.out.println("Update student information");
```

```

        System.out.println("Enter the student ID to edit: ");
        Scanner input = new Scanner(System.in);
        String rollNumber = input.nextLine();
        editInfo();
        String sql = "UPDATE `tblsinhvien` SET "
            + "`name` = '" + this.name + "',"
            + "`address` = '" + this.address + "',"
            + "`phone` = '" + this.phoneNumber + "',"
            + "`gender` = '" + this.gender + "' WHERE "
            + "`rollnumber` = '" + rollNumber + "'";
        // System.out.println("SQL: " + sql);
        int row = st.executeUpdate(sql);
        if (row > 0) {
            System.out.println("Updated successfully");
        } else {
            System.out.println("Failed, please check your data");
        }
    }
}

```

Step 3: Call it from main()

```

case 3:
    try {
        sv.update(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
    break;

```

Step 4: Run and check the result

```

Vui lòng chọn từ 1 -> 5
3
Cập nhật thông tin sinh viên
Nhập mã sinh viên muốn sửa:
JF150801
Nhập thông tin sửa:
Họ và Tên:
Giao Hop Phat 1
Địa chỉ:
Motel
SĐT:
69696969
Giới tính(Nam = 1 | Nữ = 0):
1
Sửa thành công
===== MENU =====
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5

```

Console output after choosing option 3 and updating student JF150801's information (new name "Giao Hop Phat 1", address "Motel", phone 69696969, Male).

Step 5: Write the delete() function

```

public void delete(Statement st) throws SQLException {
    System.out.println("Enter the student ID to delete: ");
    Scanner input = new Scanner(System.in);
    String rollNumber = input.nextLine();
    String sql = "DELETE FROM `tblsinhvien` WHERE "
        + "`rollnumber` = '" + rollNumber + "'";
    // System.out.println("SQL: " + sql);
    int row = st.executeUpdate(sql);
    if (row > 0) {
        System.out.println("Deleted successfully");
    } else {
        System.out.println("Failed, please check your data");
    }
}

```

Step 6: Call it from main()

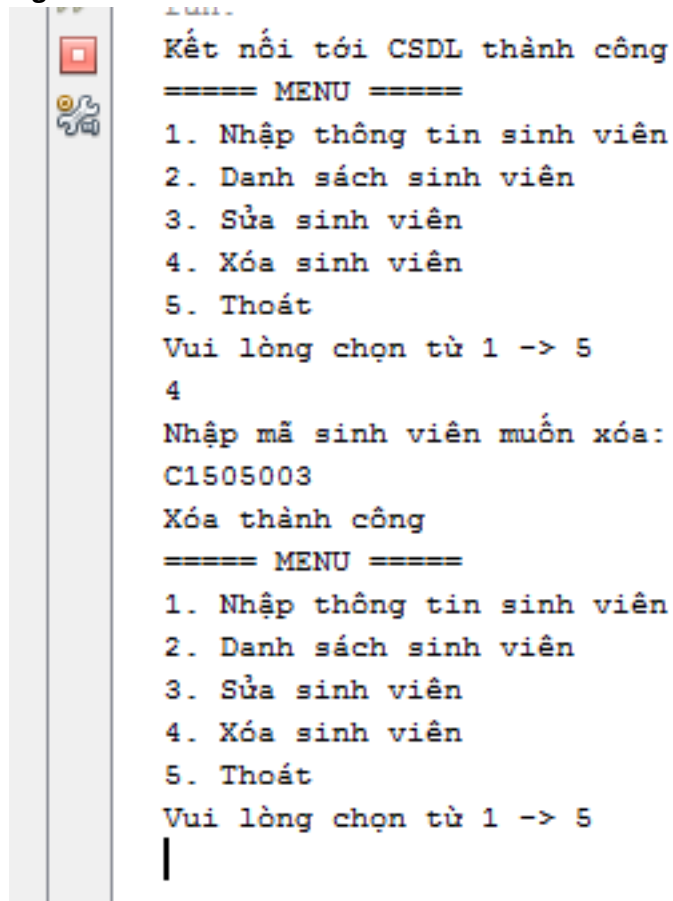
```

case 4:
    try {
        sv.delete(st);
    } catch (SQLException ex) {
        Logger.getLogger(MainClass.class.getName()).log(Level.SEVERE, null, ex);
    }
}

```

```
break;
```

Step 7: Run the program and check the result



```
Kết nối tới CSDL thành công
===== MENU =====
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5
4
Nhập mã sinh viên muốn xóa:
C1505003
Xóa thành công
===== MENU =====
1. Nhập thông tin sinh viên
2. Danh sách sinh viên
3. Sửa sinh viên
4. Xóa sinh viên
5. Thoát
Vui lòng chọn từ 1 -> 5
|
```

Console output after choosing option 4 and deleting the student with ID C1505003.